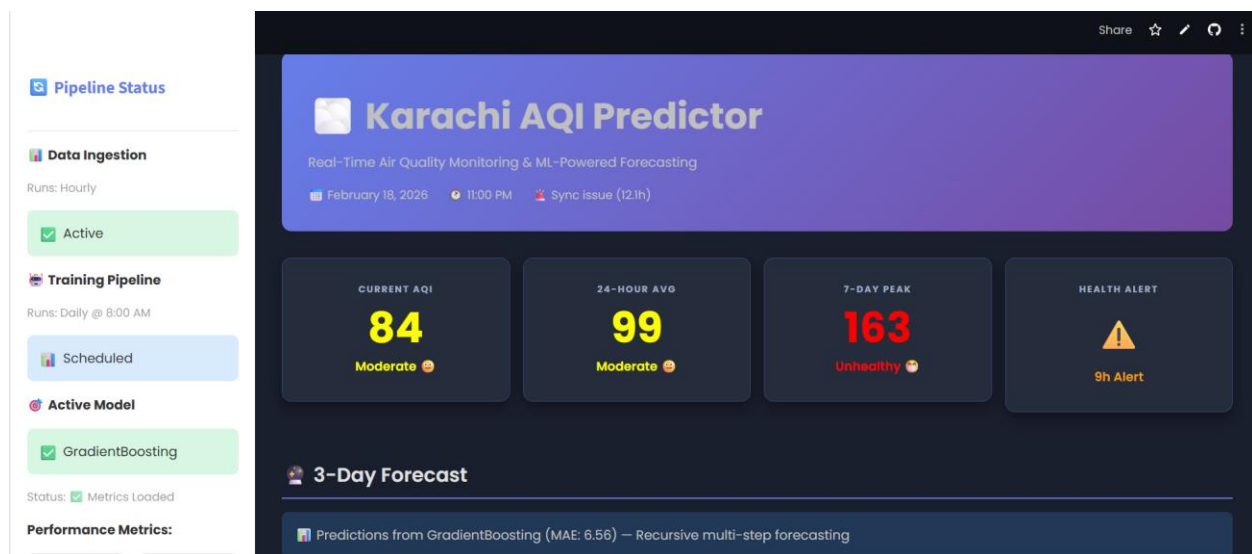


# PROJECT JOURNEY

## Karachi AQI Predictor – ML Forecasting System



## EXECUTIVE SUMMARY

This document outlines the end-to-end development journey of building a fully automated, production-grade Air Quality Index (AQI) forecasting system for Karachi, Pakistan.

The system:

- Collects hourly pollutant and weather data
- Engineers time-series features
- Retrains models daily
- Automatically selects and registers the best model
- Serves real-time 72-hour forecasts via a Streamlit dashboard
- Runs entirely on a serverless architecture using free-tier cloud tools

## Current System Status

- 9,180+ hourly data points collected
- 780+ successful hourly feature pipeline runs
- 45+ successful daily training pipeline runs
- Pipelines are fully automated and still actively running

## Best Model (Latest Run – Feb 18, 2026)

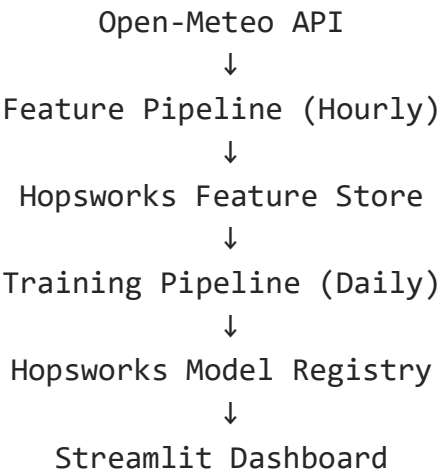
| Model             | MAE    | RMSE    | R <sup>2</sup> |
|-------------------|--------|---------|----------------|
| Linear Regression | 7.0674 | 10.8194 | 0.8736         |
| Random Forest     | 7.2140 | 10.6743 | 0.8770         |
| Gradient Boosting | 6.5632 | 9.7729  | 0.8969         |

### Selected Model: Gradient Boosting

Lowest RMSE and strongest generalization performance.

# SYSTEM ARCHITECTURE

The system follows a modular, serverless MLOps architecture:



Core Layers:

- Data ingestion
- Feature engineering
- Feature Store integration
- Automated training & evaluation
- Model Registry
- CI/CD orchestration
- Real-time inference dashboard

## Feature Pipeline (Hourly)

### Trigger

GitHub Actions (cron-based schedule)

### Data Source

Open-Meteo API (pollutants + meteorological variables)

### Storage

Hopsworks Feature Store (versioned feature groups)

### How It Works

1. Fetch hourly pollutant & weather data
2. Remove invalid or future timestamps to prevent leakage
3. Engineer structured time-series features:
  - a. Time encodings (hour, weekday, month)
  - b. Lag features
  - c. Rolling statistics
  - d. AQI change-rate features
4. Validate schema consistency
5. Store features in versioned Feature Store

## Operational Stats

- **780+ successful executions**
- Average runtime: ~1–2 minutes
- Fully automated
- Currently active and stable

The feature pipeline is idempotent and safe for retries, ensuring reliability in automated runs.

# Training Pipeline (Daily)

## Trigger

GitHub Actions (daily schedule)

## Models Implemented

- Ridge Regression
- Random Forest
- Gradient Boosting

## Training Workflow

1. Retrieve historical features from Hopsworks
2. Perform strict time-aware split (no shuffling)
3. Train multiple candidate models
4. Evaluate performance (RMSE, MAE,  $R^2$ )
5. Automatically select best-performing model
6. Save artifacts locally (artifacts/)
7. Register best model in Hopsworks Model Registry

## Latest Training Run

- Total rows: 9,180
- Gradient Boosting selected
- Model + metrics saved successfully

- Registry updated

## Operational Stats

- **45+ successful daily training runs**
- Average runtime: ~4–6 minutes
- Fully automated
- Still actively running

# Dashboard (Real-Time)

## Framework

Streamlit

## Data Flow

- Loads `model.joblib`
- Reads `latest_features.parquet`
- Displays metrics from `metrics.json`

## Dashboard Capabilities

- Current AQI display with health category
- 72-hour hourly forecast
- Historical AQI trends
- Model performance metrics
- SHAP-based feature importance
- Visual pollutant breakdown

The dashboard acts as a lightweight serving layer over the registered model and latest feature data.

# CHALLENGES ENCOUNTERED & SOLUTIONS

## Challenge 1: First-Time Integration with Hopsworks

This project was my first time working with a production-level Feature Store and Model Registry.

### Initial Complexity

- Understanding feature group versioning
- Managing schema consistency
- Integrating Feature Store queries into training workflows
- Handling authentication securely

### Outcome

By integrating Hopsworks:

- Gained real-world MLOps tooling experience
- Learned production data versioning practices
- Implemented clean separation between feature engineering and training
- Improved reproducibility of experiments

This significantly elevated the project from a notebook-based ML workflow to a production-aligned system.

## Challenge 2: CI/CD Pipeline Debugging & Stability

Early pipeline runs failed due to:

- Missing repository secrets
- Incorrect environment configuration
- Inconsistent dependency versions

## **Solution**

- Documented all required environment variables
- Added structured logging
- Pinned dependency versions
- Tested pipelines locally before pushing

## **Result**

- Stable automation
- Reliable scheduled runs
- Clear debugging trail via logs

## **Challenge 3: Model Registry Metric Validation**

Problem:

Model registration failed silently when non-numeric values were included in metrics.

Root Cause:

Hopsworks Model Registry requires strictly numeric metric dictionaries.

Solution:

Filtered metrics to ensure only numeric values are passed during registration.

Result:

Consistent and successful model version registration.

# **PIPELINE FAILURES & RESOLUTIONS**

## **Failure Type 1: Network Timeouts**

Frequency: Rare

Cause: External API response delays

Solution:

- Increased timeout thresholds
- Added retry handling

Outcome:

Pipelines now recover automatically without manual intervention.

## **Failure Type 2: Feature Group Version Conflicts**

Frequency: Development phase only

Cause:

Schema modifications without version increment

Solution:

Created new feature group versions when schema changed.

Outcome:

Maintained reproducibility and historical traceability.

# **OPTIMIZATION RESULTS**

## **Pipeline Performance Improvements**

Feature Pipeline:

- Removed redundant computations
- Optimized feature transformations
- Cached GitHub dependencies
- Reduced runtime significantly

Training Pipeline:

- Tuned Random Forest estimator count
- Reduced unnecessary re-training overhead



- Improved data loading efficiency

Dashboard:

- Implemented Streamlit caching
- Simplified rendering logic
- Reduced load time dramatically

## TECHNICAL SPECIFICATIONS

### Dataset

- 9,180+ hourly observations
- Pollutants:
  - PM2.5
  - PM10
  - Carbon Monoxide
  - Nitrogen Dioxide
  - Sulphur Dioxide
  - Ozone

### Engineered Features

- Hour, Day, Month, Weekday
- Lag features
- Rolling averages
- AQI change-rate
- Temporal encodings

### Infrastructure

- Compute: GitHub Actions (free tier)
- Storage: Hopsworks Feature Store
- Model Registry: Hopsworks

- Hosting: Streamlit Cloud
- Total Cost: \$0/month

# QUALITY ASSURANCE

## Code Quality

- Modular architecture (src/, ui/, artifacts/)
- Clear separation of ingestion, features, training
- Logging enabled
- No hardcoded credentials
- Environment variables secured

## Testing & Validation

- 780+ production feature runs
- 45+ training runs
- Holdout validation
- Manual inspection of predictions
- Data quality checks

## Documentation

- Comprehensive README
- Architecture diagrams
- Troubleshooting guide
- Clear setup instructions

# DEPLOYMENT CHECKLIST

## Streamlit Cloud

- Dependencies pinned
- Environment variables configured
- Dashboard tested locally
- Artifacts accessible

## GitHub Repository

- Clean folder structure
- CI/CD workflows active
- Secrets configured
- .gitignore configured

## Hopsworks Integration

- Feature groups versioned
- Model Registry updated
- API keys secured

# FUTURE ENHANCEMENTS

## Technical

- LSTM / Transformer forecasting
- Hyperparameter optimization (Optuna)
- Drift detection & monitoring
- Automated alert thresholds

## Product-Level

- Multi-city expansion
- Public prediction API
- SMS / Email AQI alerts
- Historical comparison dashboard

# LESSONS LEARNED

## Technical

- Production ML requires automation beyond notebooks
- Feature stores dramatically improve reproducibility
- Time-series forecasting demands strict chronological validation
- CI/CD reduces human error

## Soft Skills

- Clear logging saves hours of debugging
- Incremental iteration beats over-engineering
- Documentation reduces deployment friction

## Project Management

- Built feature pipeline first, then training, then dashboard
- Automated early instead of manually triggering runs
- Version everything (features, models, workflows)

# CONCLUSION

This project successfully demonstrates:

- A fully automated serverless ML architecture
- Reliable hourly feature computation

- Daily retraining with automated model selection
- Production-style model registry integration
- Real-time forecast serving
- Active pipelines (780+ feature runs, 45+ training runs and growing)

The system runs autonomously, handles errors gracefully, and follows modern MLOps best practices.